

深度解读GraphRAG：如何通过知识图谱提升 RAG 系统

发布日期：2024-10-16 20:39:56 浏览次数：3367 作者：Zilliz

01.

RAG 简介与 RAG 面临的挑战

检索增强生成（Retrieval Augmented Generation，RAG）是一种连接外部数据源以增强大语言模型（LLM）输出质量的技术。这种技术帮助 LLM 访问私有数据或特定领域的数据，并解决幻觉问题。因此，RAG 已被广泛用于许多通用的生成式 AI（GenAI）应用中，如 AI 聊天机器人和推荐系统。

一个基本的 RAG 通常集成了一个向量数据库和一个 LLM，其中向量数据库存储并检索与用户查询相关的上下文信息，LLM 根据检索到的上下文生成答案。虽然这种方法在大部分情况下效果都很好，但在处理复杂任务时却面临一些挑战，如多跳推理（multi-hop reasoning）或联系不同信息片段全面回答问题。

以这个问题为例：“_What name was given to the son of the man who defeated the usurper Allectus?_”

一个基本的 RAG 通常会遵循以下步骤来回答这个问题：

识别那个人：确定谁打败了 Allectus。

研究那个人的儿子：查找有关这个人家庭的信息，特别是他的儿子。

找到名字：确定儿子的名字。

通常第一步就会面临挑战，因为基本的 RAG 根据语义相似性检索文本，而不是基于在数据集中没有明确提及具体细节来回答复杂的查询问题。这种局限性让我们很难找到所需的确切信息。解决方案通常是常见查询手动创建问答对。但这种解决方案通常十分昂贵甚至不切实际。

为了应对这些挑战，微软研究院引入了 GraphRAG，这是一种全新方法，它通过知识图谱增强 RAG 的检索和生成。在接下来的部分中，我们将解释

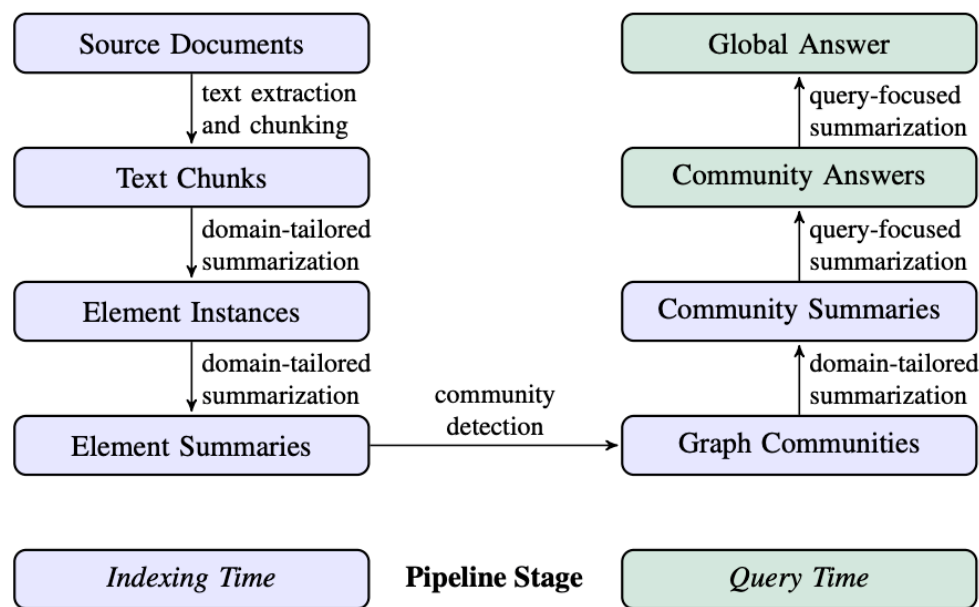
GraphRAG 的内部工作原理以及如何使用 Milvus 向量数据库搭建 GraphRAG 应用。

02.

GraphRAG 及其工作原理简介

与使用向量数据库检索语义相似文本的基本 RAG 不同，GraphRAG 通过结合知识图谱（KGs）来增强 RAG。知识图谱是一种数据结构，它根据数据间的关系来存储和联系相关或不相关的数据。

GraphRAG 流程通常包括两个基本过程：索引和查询。



(图片来源: <https://arxiv.org/pdf/2404.16130>)

索引

索引过程包括四个关键步骤：

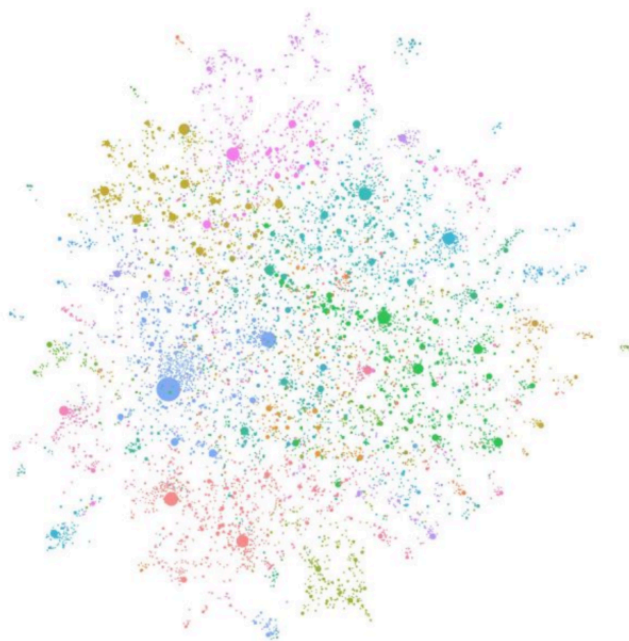
1. 文本单元分割 (Text Unit Segmentation): 整个输入语料库被划分为多个文本单元（文本块）。这些文本块是最小的可分析单元，可以是段落、句子或其他逻辑单元。通过将长文档分割成较小的文本块，我们可以提取并保留有关输入数据的更详细信息。

2. 提取 Entity、关系 (Relationship) 和 Claim: GraphRAG 使用 LLM 识别并提取每个文本单元中的所有 Entity (人名、地点、组织等)、Entity 之间的关系以及文本中表达的关键 Claim。我们将使用这些提取的信息构建初始知识图谱。

3. 层次聚类: GraphRAG 使用 Leiden 技术对初始知识图谱执行分层聚类。Leiden 是一种 community 检测算法, 能够有效地发现图中的 community 结构。每个聚类中的 Entity 被分配到不同的 community, 以便进行更深入的分析。

注意: community 是图中一组节点, 它们彼此之间紧密连接, 但与网络中其他 *dense group* 的连接较为“稀疏”。

4. 生成 Community 摘要: GraphRAG 使用自下而上的方法为每个 community 及其中的重要部分生成摘要。这些摘要包括 Community 内的主要 Entity、Entity 的关系和关键 Claim。这一步为整个数据集提供了概览, 并为后续查询提供了有用的上下文信息。



(图片来源: <https://microsoft.github.io/graphrag/>)

查询

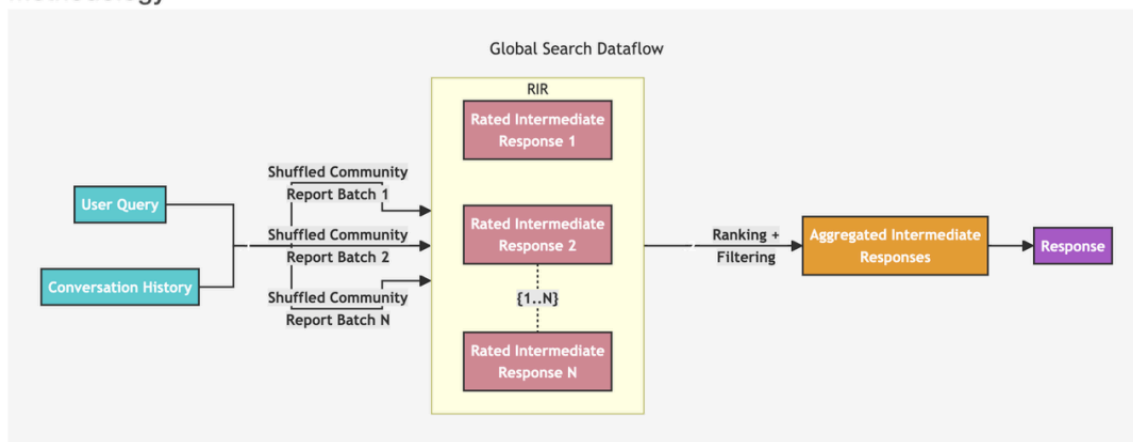
GraphRAG 有两种不同的查询工作流程, 针对不同类型的查询进行了优化:

全局搜索: 通过利用 Community 摘要, 对涉及整个数据语料库的整体性问题进行推理。

本地搜索：通过扩展到特定 Entity 的邻居和相关概念，对特定 Entity 进行推理。

这个全局搜索工作流程包括以下几个阶段：

Methodology



(图片来源: <https://microsoft.github.io/graphrag/>)

用户查询和对话历史：系统将用户查询和对话历史作为初始输入。

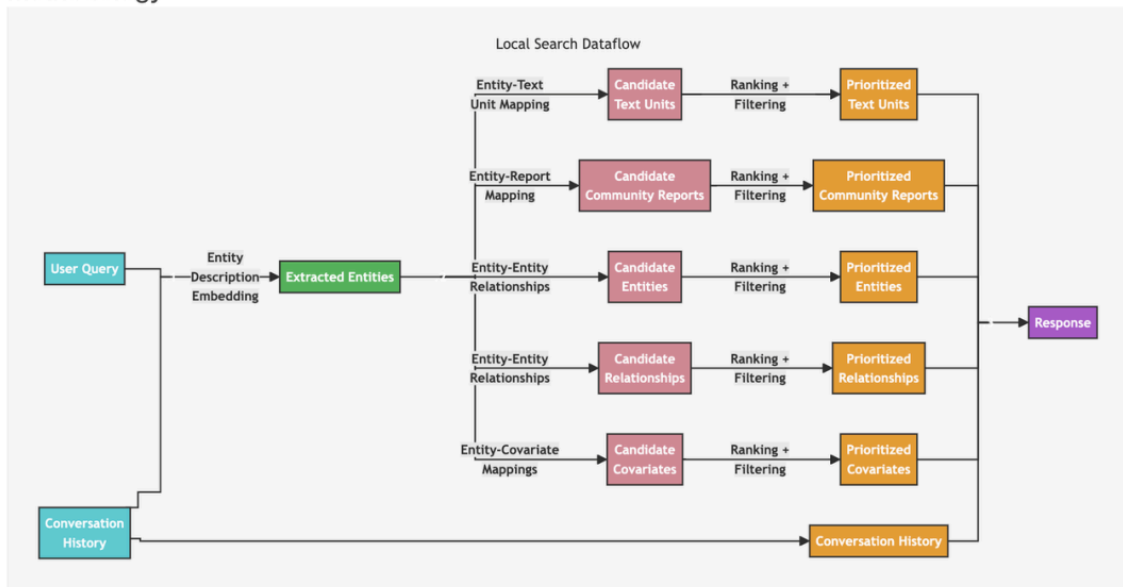
Community 报告分批：系统使用由 LLM 从 Community 层次结构的指定级别生成的节点 Community 报告作为上下文数据。这些 Community 报告被打乱并分成多个批次（打乱的 Community 报告批次 1、批次 2... 批次 N）。

RIR（评级中间响应）：每批 Community 报告进一步被划分为预定义大小的文本块。每个文本块用于生成一个中间响应。响应包含一个信息片段列表，称为点。每个点都有一个数值分数，表示其重要性。这些生成的中间响应是评级中间响应（评级中间响应 1、响应 2... 响应 N）。

排名和过滤：系统对这些中间响应进行排名和过滤，选择最重要的点。选定的重要点形成聚合的中间响应。

最终响应：聚合的中间响应被用作上下文以生成最终回复。

当用户提出关于特定 Entity（如人名、地点、组织等）的问题时，我们建议使用本地搜索工作流程。这个过程包括以下步骤：



(图片来源: <https://microsoft.github.io/graphrag/>)

用户查询：首先，系统接收用户查询，这可能是一个简单的问题或更复杂的查询。

搜索相似 Entity：系统从知识图中识别出与用户输入语义相关的一组 Entity。这些 Entity 作为进入知识图谱的入口点。这一步骤中使用像 Milvus 这样的向量数据库进行文本相似性搜索。

Entity-文本单元映射：提取的文本单元被映射到相应的 Entity，移除原始的文本信息。

Entity-关系提取：这一步提取关于 Entity 及其相应关系的特定信息。

Entity-协变量 (Covariate) 映射：这一步将 Entity 映射到它们的协变量，这可能包括统计数据或其他相关属性。

Entity- Community 报告映射：Community 报告被整合到搜索结果中，纳入一些全局信息。

利用对话历史：如果有对话历史，系统使用对话历史来更好地理解用户的意图和上下文。

生成响应：最后，系统根据前几步生成的经过过滤和排序的数据生成并响应用户查询。

03.

基础 RAG 与 GraphRAG 输出质量对比

为了展示 GraphRAG 的有效性，其开发者在博客 (<https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/>) 中比较了基础 RAG 和 GraphRAG 的输出质量。我在这里引用一个简单的例子来说明。

使用的数据集

GraphRAG 的开发者在他们的实验中使用了来自新闻文章的暴力事件信息 (Violent Incident Information from News Articles, VIINA) 数据集。

注意：此数据集包含敏感内容。选择它仅是因为它复杂，包含不同的观点和信息。这个数据集能够真实反应复杂的实际情况，且数据足够新，没有被包含在 LLM 基础模型的训练中。

实验概览

基础 RAG 和 GraphRAG 都被问到了同样的问题，这需要汇总整个数据集中的信息来构成答案。

问：What are the top 5 themes in the dataset?

下图为答案。基础 RAG 提供的结果与战争主题无关，因为向量搜索检索到了无关的文本，导致了答案的不准确。相比之下，GraphRAG 提供了一个清晰且高度相关的答案，识别了主要的主题和相关细节。结果与数据集一致，并引用了源材料。

上述例子展示了 GraphRAG 如何通过结合知识图谱和向量数据库，更有效地处理需要跨数据集整合信息的复杂查询，从而提高答案的相关性和准确性。

Query: "What are the top 5 themes in the data?"

Baseline RAG	GraphRAG
1. Improving the quality of life in cities and urban development. 2. The meeting between Vladimir Putin and the President of the United Arab Emirates, Mohammed bin Zayed Al Nahyan. 3. The National Rating of the Investment Climate in the Regions of Russia. 4. The state of the Russian economy and the growth of domestic brands. 5. The discussion on the meaning of life and the role of science in understanding it.	The data presents a wide range of themes, but the top five most prevalent themes can be identified as follows: 1. Conflict and Military Activity: A significant portion of the data revolves around the ongoing conflict in Ukraine, with numerous reports detailing military activities, occupations, and attacks by various entities. This theme includes the actions of the Ukrainian military, Russian forces, and various other military units and entities [Data: Reports (513, 241, 151, 493, 343, +more)]. 2. Political and Governmental Entities: Many reports focus on political figures and governmental entities, highlighting their roles, relationships, and actions in the context of the current situation. This includes entities like the Kremlin, the President of Ukraine, the US State Department, and various regional administrations [Data: Reports (98, 493, 265, 129, 196, +more)]. 3. Infrastructure and Environmental Concerns: Several reports highlight potential environmental impacts and infrastructure threats. These include potential environmental damage due to industrial activities, flooding crises, and threats to infrastructure due to military conflicts [Data: Reports (308, 154, 201, 348, 216, +more)]. 4. Community Analysis and Threat Assessment: Several reports provide detailed analyses of specific communities, often centered around a particular

在论文《From Local to Global: A Graph RAG Approach to Query-Focused Summarization》中进行的进一步实验表明，GraphRAG 在多跳推理和复杂信息总结方面性能明显更佳。研究表明，GraphRAG 在全面性和多样性方面都超过了基础 RAG：

- 全面性：答案覆盖问题的所有方面。
- 多样性：答案提供的观点和见解具有多样性和丰富性。

我们建议您阅读 GraphRAG 论文，以获取更多实验详情 (<https://arxiv.org/pdf/2404.16130>)。

04.

使用 Milvus 向量数据看搭建 GraphRAG 应用

GraphRAG 通过知识图谱增强了 RAG 应用。GraphRAG 依赖于向量数据库来实现检索。这一章节将介绍如何实现 GraphRAG，创建 GraphRAG 索引，并使用 Milvus 向量数据库进行查询。

前提条件

运行本文代码前，请先确保安装相关依赖：

```
pip install --upgrade pymilvus  
pip install git+https://github.com/zc277584121/graphrag.git
```

注意：我们从一个分支仓库安装了 GraphRAG，因为在撰写本文时，Milvus 的存储功能 PR 仍在等待 merge。

准备数据

从 Project Gutenberg(<https://www.gutenberg.org/>) 下载一个大约有一千行的小文本文件 (<https://www.gutenberg.org/cache/epub/7785/pg7785.txt>)，后续将用于创建 GraphRAG 索引。

这个数据集是关于达芬奇的故事集。我们使用 GraphRAG 构建与达芬奇有关的所有内容的图索引，并使用 Milvus 向量数据库搜索相关知识以回答问题。

```
import nest_asyncio  
nest_asyncio.apply()
```



```
import os
import urllib.request

index_root = os.path.join(os.getcwd(), 'graphrag_index')
os.makedirs(os.path.join(index_root, 'input'), exist_ok=True)

url = "https://www.gutenberg.org/cache/epub/7785/pg7785.txt"
file_path = os.path.join(index_root, 'input', 'davinci.txt')

urllib.request.urlretrieve(url, file_path)

with open(file_path, 'r+', encoding='utf-8') as file:
    # We use the first 934 lines of the text file, because the later lines are not relevant for this ex.
    # If you want to save api key cost, you can truncate the text file to a smaller size.
    lines = file.readlines()
    file.seek(0)
    file.writelines(lines[:934]) # Decrease this number if you want to save api key cost.
    file.truncate()
```

初始化 Workspace

现在让我们使用 GraphRAG 为文本文件创建索引。请先运行 `graphrag.index --init` 命令初始化 workspace。

```
python -m graphrag.index --init --root ./graphrag_index
```

配置 env 文件

您可以在 index 的根目录下找到 `.env` 文件。请在 `.env` 文件中添加 OpenAI 的 API key。

重要提醒:

本示例将使用 OpenAI 模型，请提前准备好 OpenAI 的 API key。

由于需要使用 LLM 处理整个文本语料库，因此 GraphRAG 索引的成本高昂。运行本示例会产生一定费用。如需节省成本，请删减本示例中的文本文件以缩减其文件大小。

运行索引 pipeline

索引过程需要一定的时间。完成后，您可以在 `./graphrag_index/output/<timestamp>/artifacts` 下找到一个新文件夹，其中包含一系列的 parquet 文件。

```
python -m graphrag.index --root ./graphrag_index
```

查询 Milvus 向量数据库

在查询阶段，我们将 Entity 描述的向量存储在 Milvus 中，用于 GraphRAG 本地搜索。这种方法结合了知识图谱中的结构化数据和输入文档中的非结构化数据，通过为 LLM 提供相关的信息来增强其上下文，从获取更精确的答案。

```

import os

import pandas as pd
import tiktoken

from graphrag.query.context_builder.entity_extraction import EntityVectorStoreKey
from graphrag.query.indexer_adapters import (
    # read_indexer_covariates,
    read_indexer_entities,
    read_indexer_relationships,
    read_indexer_reports,
    read_indexer_text_units,
)
from graphrag.query.input.loaders.dfs import (
    store_entity_semantic_embeddings,
)
from graphrag.query.llm.oai.chat_openai import ChatOpenAI
from graphrag.query.llm.oai.embedding import OpenAIEmbedding
from graphrag.query.llm.oai.typing import OpenaiApiType
from graphrag.query.question_gen.local_gen import LocalQuestionGen
from graphrag.query.structured_search.local_search.mixed_context import (
    LocalSearchMixedContext,
)
from graphrag.query.structured_search.local_search.search import LocalSearch
from graphrag.vector_stores import MilvusVectorStore

```

```

output_dir = os.path.join(index_root, "output")
subdirs = [os.path.join(output_dir, d) for d in os.listdir(output_dir)]
latest_subdir = max(subdirs, key=os.path.getmtime) # Get latest output directory
INPUT_DIR = os.path.join(latest_subdir, "artifacts")

COMMUNITY_REPORT_TABLE = "create_final_community_reports"
ENTITY_TABLE = "create_final_nodes"
ENTITY_EMBEDDING_TABLE = "create_final_entities"
RELATIONSHIP_TABLE = "create_final_relationships"
COVARIATE_TABLE = "create_final_covariates"
TEXT_UNIT_TABLE = "create_final_text_units"
COMMUNITY_LEVEL = 2

```

从索引流程中加载数据

索引流程中会生成几个 parquet 文件。我们需要将这些文件加载到内存中并将 Entity 描述信息存储在 Milvus 向量数据库中。

读取 entities:

```
# read nodes table to get community and degree data
entity_df = pd.read_parquet(f"{INPUT_DIR}/{ENTITY_TABLE}.parquet")
entity_embedding_df = pd.read_parquet(f"{INPUT_DIR}/{ENTITY_EMBEDDING_TABLE}.parquet")

entities = read_indexer_entities(entity_df, entity_embedding_df, COMMUNITY_LEVEL)

description_embedding_store = MilvusVectorStore(
    collection_name="entity_description_embeddings",
)
# description_embedding_store.connect(uri="http://localhost:19530") # For Milvus docker servi
description_embedding_store.connect(uri="milvus.db") # For Milvus Lite

entity_description_embeddings = store_entity_semantic_embeddings(
    entities=entities, vectorstore=description_embedding_store
)

print(f"Entity count: {len(entity_df)}")
entity_df.head()
```

Entity 数量: 651

level		title	type	description	source_id	community	degree	human_readable_id	
0	0	"LEONARDO DA VINCI"	"PERSON"	Leonardo da Vinci was a multi-talented genius ...	0903cfae7d96ae49cec69f8d24757557,23d7b6075d460...	6	42	0	b45241d7
1	0	"MAURICE W. BROCKWELL"	"PERSON"	Maurice W. Brockwell is the author of the eBoo...	b8f4c0d6763daa92840f358308d8dc2a,ea89c69dcfe28...	6	2	1	4119fd060
2	0	"PROJECT GUTENBERG"	"ORGANIZATION"	"Project Gutenberg is a digital library offeri...	ea89c69dcfe28e02b56ee6110db52bc4	6	2	2	d3835bf3dc
3	0	"UNITED STATES"	"GEO"	"The United States is mentioned as a location ...	ea89c69dcfe28e02b56ee6110db52bc4	6	1	3	077d2820a
4	0	"MONA LISA"	"EVENT"	The "Mona Lisa" is one of Leonardo da Vinci's ...	2e1d9a17b9ea3bea84c1434c87440a25,5bff756e44fe4...	4	6	4	3671ea0dd

读取 relationships

```
relationship_df = pd.read_parquet(f"{INPUT_DIR}/{RELATIONSHIP_TABLE}.parquet")
relationships = read_indexer_relationships(relationship_df)

print(f"Relationship count: {len(relationship_df)}")
relationship_df.head()
```

Relationship 数量: 290

	source	target	weight	description	text_unit_ids	id	human_readable_id	source_degree
0	"LEONARDO DA VINCI"	"MONA LISA"	2.0	Leonardo da Vinci is the artist who created an...	[b8f4c0d6763daa92840f358308d8dc2a, ea89c69dcfe..., b35c3d1a7daa4924b6bdb58bc69c354d		0	4
1	"LEONARDO DA VINCI"	"MAURICE W. BROCKWELL"	2.0	Maurice W. Brockwell authored the eBook "Leona...	[b8f4c0d6763daa92840f358308d8dc2a, ea89c69dcfe..., a97e2ecd870944cfbe71c79bc0fcc752		1	4
2	"LEONARDO DA VINCI"	"VINCI"	2.0	Leonardo da Vinci was born in the town of Vinc...	[8d3484980987417aee5213cec2395b3f, f680a6dc720..., 3e1b063bbfa9423d84e50311296d2f3c		2	4
3	"LEONARDO DA VINCI"	"SER PIERO"	1.0	"Leonardo da Vinci is the natural and first-bo...	[f680a6dc72077ac4c4c66e917e5e4eb5], 9a8ce816ee954bdabd01ea2081538009		3	4
4	"LEONARDO DA VINCI"	"CATERINA"	1.0	"Caterina is Leonardo da Vinci's mother."	[f680a6dc72077ac4c4c66e917e5e4eb5], 09f18f81442d4d6d93a90f0fac683f9b		4	4

读取 community reports

```
report_df = pd.read_parquet(f"{INPUT_DIR}/{COMMUNITY_REPORT_TABLE}.parquet")
reports = read_indexer_reports(report_df, entity_df, COMMUNITY_LEVEL)

print(f"Report records: {len(report_df)}")
report_df.head()
```

Report 数量: 45

	community	full_content	level	rank	title	rank_explanation	summary	findings	full_content_json	id
0	43	# Leonardo da Vinci and His Renaissance Networ...	2	9.0	Leonardo da Vinci and His Renaissance Network	The impact severity rating is high due to Leon...	This community revolves around Leonardo da Vin...	[{'explanation': 'Leonardo da Vinci is the cen...	{\n "title": "Leonardo da Vinci and His Ren...	5aa1068d-7480-45bf-b4ca-d919f8b3e060
1	44	# Francesco Sforza and Leonardo's Equestrian S...	2	7.5	Francesco Sforza and Leonardo's Equestrian Statue	The impact severity rating is high due to the ...	The community centers around the historical fi...	[{'explanation': 'Francesco Sforza is a pivota...	{\n "title": "Francesco Sforza and Leonardo...	9439df3a-c31f-402e-bb6f-98c2d6be6bd9
2	13	# Uffizi Gallery and Leonardo da Vinci Works\n...	1	8.5	Uffizi Gallery and Leonardo da Vinci Works	The impact severity rating is high due to the ...	The community revolves around the Uffizi Galle...	[{'explanation': 'The Uffizi Gallery is a cent...	{\n "title": "Uffizi Gallery and Leonardo d...	d1840658-7d14-402e-bb6f-cd606a6b2e56
3	14	# Florence and Leonardo da Vinci\n\nThe commun...	1	9.0	Florence and Leonardo da Vinci	The impact severity rating is high due to Flor...	The community revolves around Florence, a majo...	[{'explanation': 'Florence is renowned for its...	{\n "title": "Florence and Leonardo da Vinc...	7351239b-1e52-4550-a335-f98448d00995
4	15	# Pazzi Conspiracy and Key Figures\n\nThe comm...	1	8.5	Pazzi Conspiracy and Key Figures	The impact severity rating is high due to the ...	The community revolves around the Pazzi Conspi...	[{'explanation': 'Bernardo Bandini was a centr...	{\n "title": "Pazzi Conspiracy and Key Figu...	573a0c2d-f12e-4cae-95ee-b7df83ce35b1

读取 text units

```
text_unit_df = pd.read_parquet(f"{INPUT_DIR}/{TEXT_UNIT_TABLE}.parquet")
text_units = read_indexer_text_units(text_unit_df)

print(f"Text unit records: {len(text_unit_df)}")

text_unit_df.head()
```

Text unit 数量: 51

	id	text	n_tokens	document_ids	entity_ids
0	ea89c69dcfe28e056ee6110db52bc4	The Project Gutenberg eBook of Leonardo Da Vi...	300	[93341e4fe83c1fbe4d419240f35dfd17]	[b45241d70f0e43fca764df95b2b81f77, 4119fd06010...]
1	b8f4c0d6763daa92840f358308d8dc2a	Widger and the DP Team\n\n\n\n\n\n\n\n\n\n\n\n\n\nllili...	300	[93341e4fe83c1fbe4d419240f35dfd17]	[b45241d70f0e43fca764df95b2b81f77, 4119fd06010...]
2	5bff756e44fe04c0531d6de043b2ec93	anything could equal the Merit of the Man, it ...	300	[93341e4fe83c1fbe4d419240f35dfd17]	[3671ea0dd4e84c1a9b02c5ab2c8f4bac, 1fd3fa8bb55a...]
3	8d3484980987417ae5213cec2395b3f	In the National Gallery, London\n\nlV. The Las...	300	[93341e4fe83c1fbe4d419240f35dfd17]	[b45241d70f0e43fca764df95b2b81f77, f7e11b0e297...]
4	f680a6dc72077ac4ac66e917e5e4eb5	o\n\nthe main street of Vinci to-day a modern ...	300	[93341e4fe83c1fbe4d419240f35dfd17]	[b45241d70f0e43fca764df95b2b81f77, 27f9fe6ad8...]

创建本地搜索引擎

我们已经为本地搜索引擎准备了必要的数据。现在，我们可以利用这些数据、一个 LLM 和一个 Embedding 模型来构建一个 `LocalSearch` 实例。

```
api_key = os.environ["OPENAI_API_KEY"] # Your OpenAI API key
llm_model = "gpt-4o" # Or gpt-4-turbo-preview
embedding_model = "text-embedding-3-small"
```

```
llm = ChatOpenAI(
    api_key=api_key,
    model=llm_model,
    api_type=OpenaiApiType.OpenAI,
    max_retries=20,
)
```

```
token_encoder = tiktoken.get_encoding("cl100k_base")
```

```
text_embedder = OpenAIEmbedding(
    api_key=api_key,
    api_base=None,
    api_type=OpenaiApiType.OpenAI,
    model=embedding_model,
    deployment_name=embedding_model,
    max_retries=20,
)
```

```
context_builder = LocalSearchMixedContext(
    community_reports=reports,
    text_units=text_units,
    entities=entities,
    relationships=relationships,
    covariates=None, #covariates,#todo
    entity_text_embeddings=description_embedding_store,
    embedding_vectorstore_key=EntityVectorStoreKey.ID, # if the vectorstore uses entity title as
    text_embedder=text_embedder,
    token_encoder=token_encoder,
)
```

```
local_context_params = {
    "text_unit_prop": 0.5,
    "community_prop": 0.1,
    "conversation_history_max_turns": 5,
    "conversation_history_user_turns_only": True,
```

```

"top_k_mapped_entities": 10,
"top_k_relationships": 10,
"include_entity_rank": True,
"include_relationship_weight": True,
"include_community_rank": False,
"return_candidate_context": False,
"embedding_vectorstore_key": EntityVectorStoreKey.ID, # set this to EntityVectorStoreKey.T
"max_tokens": 12_000, # change this based on the token limit you have on your model (if yo
}

llm_params = {
    "max_tokens": 2_000, # change this based on the token limit you have on your model (if you
    "temperature": 0.0,
}

```

```

search_engine = LocalSearch(
    llm=llm,
    context_builder=context_builder,
    token_encoder=token_encoder,
    llm_params=llm_params,
    context_builder_params=local_context_params,
    response_type="multiple paragraphs", # free form text describing the response type and for
)

```

进行查询

```

result = await search_engine.asearch("Tell me about Leonardo Da Vinci")
print(result.response)

```

Leonardo da Vinci

Leonardo da Vinci, born in 1452 in the town of Vinci near Florence, is widely celebrated as on

Early Life and Training

Leonardo's early promise was recognized by his father, who took some of his drawings to Anc

Artistic Masterpieces

Leonardo is perhaps best known for his iconic paintings, such as the "Mona Lisa" and "The L

Scientific and Engineering Contributions

Leonardo's genius extended beyond art to various scientific and engineering endeavors. He n


```
## Patronage and Professional Relationships
```

Leonardo's career was significantly influenced by his patrons. Ludovico Sforza, the Duke of M

```
## Legacy and Influence
```

Leonardo da Vinci's influence extended far beyond his lifetime. He founded a School of painti

In summary, Leonardo da Vinci's unparalleled contributions to art, science, and engineering, c

GraphRAG 的结果非常具体，引用的数据源都清楚地标记了出来。

生成推荐问题

GraphRAG 还可以根据历史查询生成问题，这在创建聊天机器人的推荐问题时非常有用。这种方法结合了知识图谱中的结构化数据和输入文档中的非结构化数据，以产生与特定 Entity 相关的问题。

```
question_generator = LocalQuestionGen(  
    llm=llm,  
    context_builder=context_builder,  
    token_encoder=token_encoder,  
    llm_params=llm_params,  
    context_builder_params=local_context_params,  
)
```

```
question_history = [  
    "Tell me about Leonardo Da Vinci",  
    "Leonardo's early works",  
)
```

基于查询历史生成推荐问题。

```
candidate_questions = await question_generator.agenerate(  
    question_history=question_history, context_data=None, question_count=5  
)  
candidate_questions.response
```

```
["- What were some of Leonardo da Vinci's early works and where are they housed?",  
"- How did Leonardo da Vinci's relationship with Andrea del Verrocchio influence his early work?",  
"- What notable projects did Leonardo da Vinci undertake during his time in Milan?",  
"- How did Leonardo da Vinci's engineering skills contribute to his projects?",  
"- What was the significance of Leonardo da Vinci's relationship with Francis I of France?"]
```

如需删除 index 来节省空间，您可以删除 index root。

```
# import shutil  
  
#  
  
# shutil.rmtree(index_root)
```

05.

总结

本文探索了 GraphRAG —— 这是一种通过整合知识图谱来增强 RAG 技术的创新方法。GraphRAG 非常适合处理复杂任务，如多跳推理（multi-hop reasoning）和联系不同信息片段全面回答问题等。

与 Milvus 向量数据库结合使用时，GraphRAG 可以驾驭大型数据集中复杂的语义关系，提供更准确的结果。这种强强联合使 GraphRAG 成为各种实际 GenAI 应用中不可或缺资产，为理解和处理复杂信息提供了强大的解决方案。

分享：

53AI，企业落地大模型首选服务商

产品：场景落地咨询+大模型应用平台+行业解决方案

承诺：免费POC验证，效果达标后再合作。零风险落地应用大模型，已交付160+中大型企业

上一篇：探索DB-GPT V0.6.0：构建智能数据助手应用开发指南

下一篇：LightRAG为什么好用又便宜

返回列表

友情链接：[通往AGI之路](#) [云臻信息](#) [企微SCRM](#) [小名片](#) [优网科技](#)

CopyRight © 2012-2024 深圳市博思协创网络科技有限公司 版权所有  粤ICP备14082021号

广州：广州市华景路37号(华景软件园)暨南大学科技大厦6楼（整层）

深圳：深圳市福田区泰然四路29号天安创新科技广场一期A座1204

上海：上海市浦东新区金新路58号1602室



微信扫码
和创始人交个朋友